HANDOVER — MiniCPM-V 4.6 on-device extraction, latency + accuracy investigation

Written 2026-09-30. **This supersedes the root `HANDOVER.md`**, which was written on a *different machine* (`/home/kaarunya/Downloads/model_testing`) while that host's root disk was stuck in `emergency_ro`. Several of its claims are now stale — most importantly it says MiniCPM-V "needs no code change", which is no longer true. Read this file first; consult the old one only for background on the llama.cpp fork and the OpenMP investigation.

---

1. What this project is

An Android app that extracts structured fields from photos of Indian identity documents (Aadhaar, PAN, DL, Passport) by running a small vision-language model **entirely on-device on the CPU**, via a forked llama.cpp. It is a research / benchmarking tool, not a shipping product.

- **App source:** `home/aiteam/model_testing/android-app/` — the whole app is one file, `app/src/main/java/com/modeltesting/vlmtester/MainActivity.kt` (~2000 lines)
- **App build:** `app/build.gradle.kts` — AGP 8.5.2, Kotlin 1.9.24, compileSdk 35, minSdk 28, `useLegacyPackaging = true`
- **Prompts/grammars:** `app/src/main/assets/prompts/*.txt`, `assets/schemas/*.json`
- **Inference engine:** `home/aiteam/model_testing/qvac-visionpsy-nano/llama-cpp-inference/llama.cpp-custom/`
- **Benchmarks/docs at repo root:** `run_log.txt`, `LATENCY_OPTIMIZATION.md`, `QWEN3_VL_TWEAKS_LOG.md`, `terminal_report.py`, `*_bench_results/`

Repo quirk: `.gitignore` lists `home/`, but 2,144 files under it are already tracked, so edits there DO show in `git diff`. The ignore rule only affects new files. Do not assume the app source is untracked.

Target device: Samsung Galaxy M35 5G (`SM-M356B`, `RZCXA1SXH5V`), Exynos 1380 (4× A78 "big" + 4× A55 "little"), Mali-G68 MP5, ~5.4 GB usable RAM.

---

2. Current state — read this before changing anything

Uncommitted code change (the only work in flight)

`MainActivity.kt` is modified: **+157 / −18**. It is **written but never compiled** — this host has no JDK, Gradle, or Android SDK. Treat it as unverified.

What it contains:

1. **Run-log attribution** — every run now records `model=`, `mmproj=`, `threads=`. Previously the model was never logged, so historical runs cannot be attributed to a model. Several claims in `HANDOVER.md` and `LATENCY_OPTIMIZATION.md` rest on unattributable runs.
2. **Per-pass diagnostics** (`PassResult.diag`):
   - CLI: image dimensions + llama.cpp's own `prompt eval time` / `eval time` split

    - Server: image dims, `server=COLD/WARM`, cold-start seconds, true `total_wall=`, and `prompt_tokens` / `completion_tokens` from the response `usage` block
    - **Note:** the deployed `libllama-mtmd-cli.so` does **not** emit `llama_perf_context_print` lines, so the CLI half degrades to `perf: not reported by binary`. The `usage` half works and is the useful one.
3. **Measurement fix:** the server's per-request timer previously started *after* `ensureServerRunning()`, so every logged `server mode | 102.8s` entry was request time only — the model load was never counted anywhere. Both numbers are now shown.
4. **`--no-warmup`** added to the CLI args (measured −4.1s, byte-identical output).
5. **Server restart fix:** `imageTokenCap` was part of the server reuse check, but it is applied as a *startup* env var, so toggling pan_full's 128/256 switch tore down a running server and paid a full model reload. Now gated by `capAffectsServer()` — only models that actually read the cap restart on change.
6. **Two-sided MiniCPM downscale fix:** the front/back passes never called `downscaleForMiniCpm`, so `tt_id`/`dl` sent full-resolution images straight into the 9-tile ceiling. Both sides now downscale.
7. **Latent format-string bug fix:** `$cleaned`/`$report` (model-derived text) were interpolated into a `String.format` template, so a literal `%` in any extracted field would corrupt the saved run. Header is now formatted separately and concatenated.

Environment blockers

- **No build toolchain on this host** (no JDK / Gradle / Android SDK). Nothing can be compiled here. Getting a working APK requires installing them.
- `app/src/main/jniLibs/` **does not exist** in the source tree. Per `build.gradle.kts` the native `.so` files must live there to be packaged. The already-built `app_build/base.apk` *does* contain all 11 of them (verified), so the existing APK is self-contained — but **any rebuild must first copy `app_build/lib/arm64-v8a/*.so` into `app/src/main/jniLibs/arm64-v8a/`** or the APK ships with no llama.cpp binaries and fails at the "Missing native binary" check.
- Models are **not** in the APK. They are pushed to `/sdcard/Android/data/com.modeltesting.vlmtester/files/models/`. All 12 are already on the device, including Qwen3-VL-2B.

---

3. What was measured this session

All numbers: MiniCPM-V 4.6, `minicpm-v-4.6-q4_k_m.gguf` (529 MB LM) + `mmproj-minicpm-v-4.6-q8_0.gguf` (728 MB projector), CPU-only, 4 threads, `-fa on`, doc type `pan_full`, grammar-constrained. **No other model was run.**

Latency

Path	Time
CLI, 560px — the app's default	58.6 / 60.0 / 62.1 / 61.8s → **~60.6s avg**
CLI + `--no-warmup`	**54.5s** (−4.1s, byte-identical)
CLI with `-n 1`	56.6s → **JSON generation is only ~2s**
Server model load (cold start)	**24.6s**

Path	Time
Server, first uncached request	47.6–49.9s
Server, identical repeat	**7.1s** (prompt cache, 528/532 tokens)

The 7.1s vs ~48s gap **explains the bimodality in `run_log.txt`**: warm-cache requests are ~7s, uncached ones ~48s, and the old log could not tell them apart.

The cost model (this is the key structural finding)

Vision encode cost steps **discretely** at the 448px slice threshold, not smoothly:

Longest edge	prompt_tokens	Time
≤448px (1 encode)	404	~25–27s
560px (2 encodes)	532	~48–56s
900px (3 encodes)	666	~67–73s

~+22s per additional 448×448 tile. Root cause is in the forked llama.cpp: MiniCPM-V maps to `PROJECTOR_TYPE_MINICPMV4_6` → `mtmd_image_preprocessor_llava_uhd` (the minicpmv branch, `mtmd.cpp:441-451`), which slices whenever `max(w,h) > hparams.image_size` (448). The `mmproj` is Q8_0 at 728 MB — **larger than the LM itself** — and that is what each tile costs.

Two consequences that invalidate earlier project assumptions:

- `MTMD_MAX_LONGEST_EDGE` is read only by the *idefics3* preprocessor (`mtmd-image.cpp:1086`), and `LLAMA_ARG_IMAGE_MAX_TOKENS` only by *dyn_size* (Qwen). The `llava_uhd`/minicpmv slicer reads **neither**. So the Stage-3 tile fix from `LATENCY_OPTIMIZATION.md` and the app's image-token-cap toggle are both **inert for MiniCPM-V**. The only resolution lever is the Kotlin-side downscale.
- The `downscaleForMiniCpm(560)` cap in `MainActivity.kt:429` was chosen to hit 1-2 tiles and was never validated for legibility. It turns out to be roughly the right value — see below.

Accuracy

Tight crops (4 cards from `pan_test_images/`, landscape 600×315 – 860×553):

Resolution	Fully correct
560px	**4/4**
448px	1/4
420px	1/4
336px	1/4
224px	PAN digit wrong
112px	PAN not read at all

Failure mode below 448 is `parent_name` and `dob` coming back **null**, and it is *unstable* — a different card fails at each setting, so you get confident-looking but silently wrong output.

Realistic camera-photo simulation (card occupying ~65% of a 1080×1397 frame — this is the caveat that matters, since the four test images are tight crops):

Config	Fully correct
photo @560px (what the app does today)	**2/4**
cropped @448px	1/4
photo @900px	2/4 — but a *different* card fails

The 4/4-at-560 result is an artifact of tight crops. On realistic input the app's current setting fails on half the cards. Raising resolution to 900px fixed card_d while *breaking* card_c's PAN — accuracy is **not monotonic** in resolution.

---

4. Hypotheses tested and DISPROVEN — do not retry these

- **Thread placement / OpenMP pinning.** The old `HANDOVER.md` §4 claimed the deployed `.so` was built with `GGML_OPENMP=ON` and therefore never applies the `-Cr 4-7` cpumask, making the core-pinning a no-op. Tested properly with 8 samples per config over 2 rounds: baseline median **50.0s / 53.1s** vs `OMP_PLACES=4,5,6,7 OMP_PROC_BIND=close` **50.4s / 53.2s**. **No effect.** Also: `OMP_PLACES=cores` + `OMP_PROC_BIND=close` is catastrophic (**191s**, looks like all threads serialised on one core), and 8 threads is worse (**66.1s**). Conclusion: the vision encode is compute-bound and already near the hardware limit. **Do not ship OMP env vars.**
- **ROI-crop to the card then cap at 448px.** Intended to get 1 encode *and* higher effective card resolution. Result: **1/4 correct**, worse than doing nothing. The accuracy threshold and the 448px slice threshold sit almost on top of each other, so 1 encode and sufficient legibility are mutually exclusive with this slicer.
- **Lowering `downscaleForMiniCpm` toward 504px** (an earlier suggestion in this session). Would have saved ~23s and corrupted `parent_name`/`dob` on most cards.
- **"CLI vs server causes the accuracy loss."** Runs 199 (CLI, 5/5) and 200 (server, 2/5) on the same card looked like a smoking gun. Controlled test at 560px showed both paths produce identical correct output. **Theory withdrawn.**

---

5. Open problems, ranked

1. **There is no ground truth.** Every "correct" label in §3 is *inferred* from agreement between configurations, not from a real answer key. Concrete contradictions: card_c returned three different PAN numbers at three resolutions (`FLCPM0001F` / `FCLPM0001F` / `FCLPM0017F`); card_d read `THEAS1021P` on the CLI path and `AJTPI4524M` on the server path. **Any accuracy claim is provisional until a ground-truth file exists.**
2. **Accuracy on real camera photos is the binding constraint, not latency.** The app as configured gets 2/4 on realistic input. This outranks all speed work.
3. **The `run_log.txt` null-field failures are still unexplained** — runs 192–194 and 200 show null `parent_name`/`parent_relation`/`dob`. The signature matches the sub-448px degradation pattern,

but the app always used a 560px cap, so that does not account for it.

4. **Qwen3-VL-2B has never been benchmarked** on any of these images, despite being on the device. If accuracy is model-bound rather than resolution-bound, this is a bigger lever than anything left in tuning.
5. **The `mmproj` has never been requantized.** It is Q8_0 / 728 MB and dominates per-tile cost. Q4_K_M (~400 MB) is the only untested change that reduces *per-tile* cost rather than tile count. Requires a working `llama-quantize`; the fork has no build tree or NDK on this host.

---

6. Reusable test harness (on the device, still in place)

The app was never rebuilt or reinstalled. Everything ran through the native binaries directly via `adb shell`, which is how the earlier MiniCPM work was done and how you should continue.

```
/data/local/tmp/mini/          8 .so files (from app_build/lib/arm64-v8a/)
/data/local/tmp/mini/mt/       prompts, schema, test images, runner scripts
```

Start a server (waits for the readiness line, reports cold start):

```
adb shell "cd /data/local/tmp/mini/mt && ./screen.sh <label> <threads>
[ENV=V ...]"
adb forward tcp:8099 tcp:8099
```

- `run.sh <img> <n> <label> [flags]` — replicates `runOnePass()`'s exact CLI args
- `screen.sh` — starts `llama-server` with configurable threads/env
- `bench.py <label> [cards]` — times warm requests on `vreq_<card>_560.json`
- `ab.py <label> <n>` — repeated timing on `var0..varN.json`

Timing gotchas that will waste your time if you forget them:

- Android's `mksh` does **32-bit arithmetic**. `S=$(date +%s%N)` then `$(( (E-S)/1000000 ))` silently overflows and reports ~2s for a 50–90s run. Use `date +%s.%N` with `awk` for the delta. (This produced a full round of garbage numbers before it was caught.)
- The server's **prompt cache** returns **7.1s** for an identical repeat. To take a genuine measurement, vary the input — e.g. re-encode the same image at different JPEG qualities to get distinct prefixes.
- The **first request after a server restart is an outlier** (56–63s vs ~50s steady state). Discard it or use medians.
- `adb forward` does not survive; re-issue it after every `adb shell` reconnect.

Reference for native invocation: the flags in `run.sh` mirror `MainActivity.runOnePass()` (grammar `-jf`, `--temp 0`, `--repeat-penalty 1.1`, `-t/-tb`, `-Cr 4-7 -Crb 4-7 --cpu-strict`, `-fa on`, `-ngl 0 --no-mmproj-offload --no-op-offload`).

---

7. Suggested order of work

1. **Build a ground-truth file** for the test cards. Without it nothing else can be scored. Also zoom in and settle card_c's and card_d's real PAN numbers by eye.
2. **Re-run the resolution matrix on real camera photos** (not tight crops) with that ground truth, to find the true accuracy/coverage point. The current 560px cap was tuned on crops and is likely mis-tuned for real input.
3. **Benchmark Qwen3-VL-2B on the identical images** with the same harness. Only change `-m`/`--mmproj` in `run.sh`/`screen.sh`. If it wins, the model is the constraint and the latency work is moot.
4. Only then revisit latency. Realistic target is ~48–56s warm; the path below that is currently closed.
5. Optionally: requantize the `mmproj` to Q4_K_M. Highest-value untested lever, but needs `llama-quantize` and an NDK.

Do **not** spend more time on OpenMP/thread pinning or on cropping to 448px — both are measured and disproven.